NEXUS — Manual técnico

SECCIÓN I. ARQUITECTURA Y PROGRAMACIÓN ORIENTADA A OBJETOS

1. Propósito y Funcionamiento General de NEXUS

El sistema **NEXUS** ha sido concebido como un entorno de simulación dinámico y modular en C# / .NET que coordina entidades biológicas, mecánicas y virtuales dentro de una infraestructura convergente. Su arquitectura responde a la necesidad de gestionar interactividad en tiempo real, persistencia de estados de entidades, enlaces de sincronización de datos y representaciones espaciales en la interfaz operativa IRIS.

El objetivo central del núcleo operativo es proveer una plataforma extensible, escalable y de alta mantenibilidad mediante la aplicación rigurosa del paradigma de **Programación Orientada a Objetos (POO)** en C#. Esto permite aislar responsabilidades, garantizar la integridad de las variables de estado y facultar la incorporación de nuevos módulos sin comprometer la estabilidad del sistema base.

---

2. Estructura del Proyecto y Responsabilidad de sus Clases

El proyecto se organiza mediante los siguientes archivos fuente C# (`.cs`) en la raíz del repositorio:

Archivo	**Espacio de Nombres (Namespace)**	**Descripción del Contenido y Responsabilidad**
`Personaje.cs`	`Nexus.Core.Entities`	Clase base abstracta que define el contrato fundamental y los atributos/comportamientos comunes de las entidades vivas (salud, posición, nombre).
`Cadete.cs`	`Nexus.Core.Entities`	Especialización de `Personaje` para usuarios/unidades en

Archivo	Espacio de Nombres (Namespace)	Descripción del Contenido y Responsabilidad
		entrenamiento. Maneja nivel, experiencia, rango y habilidades específicas.
Enemigo.cs	`Nexus.Core.Entities`	Especialización de `Personaje` que implementa lógica de hostilidad, niveles de amenaza y patrones de ataque reactivos.
Robot.cs	`Nexus.Core.Entities`	Clase para unidades sintéticas autónomas; maneja estados de batería, directivas y protocolos de soporte.
Inventario.cs	`Nexus.Core.Items`	Contenedor dinámico estructurado (`List<Objeto>`) para la gestión, almacenamiento y filtrado de objetos del sistema.
Objeto.cs	`Nexus.Core.Items`	Definición base de artefactos, herramientas y consumibles manipulables dentro del entorno (peso, valor, tipo).
EnlaceSincronico.cs	`Nexus.Core.Network`	Gestor de comunicaciones en tiempo real y transmisión de telemetría de estado entre entidades.
MapaRealidad.cs	`Nexus.Core.World`	Entorno de representación cartesiana y topológica de sectores, zonas de colisión y posiciones de entidades.
ConsolaIris.cs	`Nexus.UI.Terminal`	Interfaz de terminal/consola encargada de capturar entrada de comandos y

Archivo	Espacio de Nombres (Namespace)	Descripción del Contenido y Responsabilidad
		renderizar respuestas formateadas.
`Program.cs`	`Nexus.App`	Punto de entrada ejecutable (`Main`) que inicializa los subsistemas e invoca el bucle principal de ejecución.

---

3. Tabla Detallada de Acoplamiento y Relaciones entre Clases

Clase Origen	Clase Destino / Colaboradora	Tipo de Relación	Descripción del Acoplamiento y Parámetros Intercambiados
`Program`	**`ConsolaIris`**, **`MapaRealidad`**, **`EnlaceSincronico`**	Instanciación / Coordinación	`Program` instancia los subsistemas principales en el `Main()` y los invoca secuencialmente pasando parámetros de configuración inicial.
`Cadete`	**`Personaje`**	Herencia (`:`)	`Cadete` hereda todos los atributos protegidos (`salud`, `posicion`, `nombre`) y métodos base de `Personaje`, extendiéndolos con campos propios de progresión.
`Enemigo`	**`Personaje`**	Herencia (`:`)	`Enemigo` hereda la estructura base de `Personaje` y sobrescribe o

Clase Origen	Clase Destino / Colaboradora	Tipo de Relación	Descripción del Acoplamiento y Parámetros Intercambiados
			implementa los comportamientos de ataque y hostilidad.
Robot	**Personaje**	Herencia / Asociación	Unidades sintéticas que comparten el comportamiento operativo básico de los actores del sistema.
Cadete	**Inventario**	Composición (1 a 1)	Cada `Cadete` posee una instancia exclusiva de `Inventario`. Si el cadete es destruido o liberado, su inventario asociado también deja de existir.
Inventario	**Objeto**	Agregación / Colección (1 a *)	`Inventario` administra una lista genérica `List<Objeto>`. Los objetos pueden existir de forma independiente dentro del mundo antes o después de ser almacenados.
EnlaceSincronico	**Personaje**	Asociación / Uso	`EnlaceSincronico` recibe referencias a objetos `Personaje` como parámetros en sus métodos (ej. `Sincronizar(Pers`

Clase Origen	Clase Destino / Colaboradora	Tipo de Relación	Descripción del Acoplamiento y Parámetros Intercambiados
			`onaje p1, Personaje p2)` para actualizar sus estados en tiempo real.
`MapaRealidad`	**`Personaje`**	Asociación / Agregación	`MapaRealidad` mantiene colecciones espaciales de las entidades y evalúa coordenadas mediante vectores, actualizando la posición de los personajes.
`ConsolaIris`	**`Program`** / Subsistemas	Dependencia de I/O	`ConsolaIris` captura comandos textuales ingresados por el operador y retorna cadenas o resultados procesados al flujo principal de ejecución.

---

4. Comparativa: Programación Estructurada vs. POO

Criterio / Cuestión	Programación Estructurada	Programación Orientada a Objetos (POO)
Organización Central	Basada en funciones, procedimientos y algoritmos secuenciales.	Basada en objetos que combinan estado (datos) y comportamiento (métodos).

Criterio / Cuestión	Programación Estructurada	Programación Orientada a Objetos (POO)
Tratamiento de Datos	Los datos son pasivos y globales o locales a rutinas; separados de las funciones.	Los datos están encapsulados dentro de los objetos y protegidos de accesos indebidos.
Reutilización de Código	Mediante librerías de funciones o llamadas a subrutinas repetitivas.	Mediante **Herencia** (relación es-un) y **Composición** (relación tiene-un).
Mantenibilidad y Escala	Difícil en proyectos grandes; los cambios en estructuras de datos rompen funciones.	Alta; el diseño modular limita los impactos del cambio al ámbito de cada clase.
Modelado de la Realidad	Convierte los procesos del mundo real en diagramas de flujo algorítmicos.	Modela entidades del mundo real como objetos con estados, identidades y roles.
Polimorfismo	Ausente o simulado complejo mediante condicionales múltiples (`switch/case`).	Nativo mediante punteros VTable, interfaces y sobrescritura de métodos (`virtual`/`override`).
Aplicación en NEXUS	Provocaría un código monolítico e inmanejable para coordinar entidades complejas.	Permite tratar a `Cadete`, `Enemigo` y `Robot` homogéneamente como `Personaje`.

SECCIÓN II. FUNDAMENTOS DE CLASES Y OBJETOS

1. Clases, Objetos e Instanciación

- **Clase:** Plantilla o molde que especifica las propiedades y comportamientos de los objetos. En NEXUS, la clase `Objeto` (en `Objeto.cs`) sirve de molde para los ítems.
- **Objeto:** Instancia concreta en memoria creada a partir de una clase.
- **Instanciación:** Proceso de reservar memoria utilizando el operador `new`. Ejemplo en `Program.cs`: `Objeto escudo = new Objeto("Escudo de Plasma", 150.0, "Defensa");`

2. Atributos, Campos, Propiedades y Métodos

- **Campos:** Variables privadas que almacenan el estado interno.
- **Propiedades:** Mecanismo con accesores `get` y `set` para exponer campos controlando su acceso.
- **Métodos:** Bloques de código que definen las acciones del objeto (ej. método `Usar` en `Objeto.cs`).

3. Constructores y Sobrecarga

- **Constructor:** Método especial que inicializa el objeto al instanciarlo con `new`.
- **Sobrecarga:** Definir múltiples constructores con diferente número o tipo de parámetros (ej. constructores básicos y completos en `Cadete.cs`).

4. Encapsulamiento y Modificadores de Acceso

- Ocultamiento de la implementación interna. En NEXUS se emplean: `public` (acceso global), `private` (acceso exclusivo de la clase), `protected` (acceso para clases derivadas) e `internal`.

5. Uso de `this` y `base`

- **`this`:** Referencia a la instancia actual de la clase para evitar ambigüedades.
- **`base`:** Referencia a la clase padre para invocar sus constructores o métodos desde clases derivadas (ej. en `Cadete.cs` llamando al constructor de `Personaje`).

SECCIÓN III. HERENCIA, POLIMORFISMO E INTERFACES

1. Clases Base y Clases Derivadas (Herencia)

Permite reutilizar código mediante la relación ES-UN. `Personaje` actúa como base, mientras que `Cadete`, `Enemigo` y `Robot` heredan su estructura y atributos base.

2. Métodos `virtual` y `override`

- **`virtual`:** Permite que un método de la clase base (`Personaje.cs`) pueda ser reescrito.
- **`override`:** Permite que la clase derivada (`Cadete.cs`) implemente su versión específica del método.

3. Clases y Métodos `abstract`

- **Clase abstracta:** No se puede instanciar directamente; sirve exclusivamente de contrato base.
- **Método abstracto:** Obliga a las clases derivadas a implementar su propia lógica mediante `override` (ej. `Accionar()` en `Personaje.cs`).

4. Polimorfismo

Capacidad de tratar objetos de diferentes clases derivadas de manera uniforme a través de una referencia base (`List<Personaje>`), ejecutando la versión correcta del método en tiempo de ejecución.

5. Interfaces y Contratos

- **Interfaces:** Contratos puros sin implementación. *Ejemplo complementario:* `ISincronizable` implementado en clases del sistema para unificar comportamientos transversales.

6. Diferencias entre Herencia, Composición e Interfaces

- **Herencia:** Relación ES-UN (`:`), reutiliza estado y estructura.
- **Composición:** Relación TIENE-UN, ciclo de vida acoplado (ej. `Cadete` contiene `Inventario`).
- **Interfaces:** Contrato de comportamiento puro sin herencia de estado.

SECCIÓN IV. COLECCIONES Y LINQ

1. Listas Genéricas `List<T>`

Colecciones dinámicas utilizadas en `Inventario.cs` (`List<Objeto>`) para almacenar y administrar elementos sin tamaño fijo.

2. Expresiones Lambda

Sintaxis concisa con operador `=>` para consultas y métodos anónimos (ej. `_items.Any(x => x.Nombre == "Botiquín")`).

3. Consultas LINQ

Uso de métodos como `Where`, `Select` y `OrderByDescending` para filtrar, transformar y ordenar colecciones de forma declarativa.

4. Comparativa: LINQ vs. Recorridos Tradicionales

- **LINQ:** Código más declarativo, limpio y legible.
- **Recorridos tradicionales (`foreach`):** Mayor control manual del bucle y depuración paso a paso.

SECCIÓN V. ANÁLISIS DEL FUNCIONAMIENTO

1. Flujograma del Proceso de Ejecución

[Inicio Program.Main()] ──► [Instanciar Subsistemas (ConsolaIris, MapaRealidad)] ──► [Cargar Entidades] ──► [Bucle de Comandos Consola] ──► [Ejecución Polimórfica p.Accionar()] ──► [Fin]

2. Seguimiento del Estado de un Objeto

- **Instancia:** `Cadete c = new Cadete("Alex", 100);` ➔ Salud: 100, Rango: "Recluta".
- **Acción:** `c.RecibirDaño(30);` ➔ Salud: 70.
- **Acción:** `c.GanarExperiencia(150);` ➔ Experiencia: 150, Rango actualizado.

3. Predicción de Resultado de Código

- Lista polimórfica de `Personaje` con un `Cadete` y un `Enemigo` aplicando `RecibirDaño(20)`. Resultado esperado en consola: impresión de los estados reducidos en 20 puntos para cada entidad de forma correcta.

4. Análisis de Modificación de Condiciones

- Modificar el modificador de acceso de `salud` en `Personaje.cs` de `protected` a `private` genera un error de compilación inmediato en las clases derivadas (`Cadete`, `Enemigo`), ya que pierden acceso directo al campo heredado.

SECCIÓN VI. PREPARACIÓN PARA EL EXAMEN H3

1. **Comprensión de POO:** Diferencia entre clase y objeto, y por qué la herencia exige una relación semántica estricta ES-UN.
2. **Análisis de Clases y Objetos:** Ventajas de usar propiedades con `{ get; private set; }` para proteger el encapsulamiento frente a campos públicos.

3. **Herencia y Polimorfismo:** Consecuencias de omitir la implementación de un método `abstract` en una clase derivada (error de compilación).
4. **Interfaces y Diseño:** Criterio de selección entre clases abstractas (estado + contrato) e interfaces (solo contrato sin estado).
5. **Listas y LINQ:** Construcción de una consulta LINQ para filtrar y ordenar objetos en C#.
6. **Predicción de Código:** Identificación de errores al intentar instanciar una clase abstracta directamente con `new`.

SECCIÓN VII. RESULTADO DEL PROMPT ESPEJO

1. Fortalezas demostradas

- **Gestión de control de versiones:** Demostraste iniciativa y comprensión al configurar un entorno de repositorio en GitHub (`ekaitzyale/NEXUS`), estructurar carpetas organizadas (`docs/`), y entender la necesidad de mantener el control físico de los archivos locales frente a las interfaces de sincronización en la nube.
- **Toma de decisiones arquitectónicas:** Supiste evaluar y proponer una solución práctica al decidir sustituir diagramas visuales complejos o automatizados por una **tabla detallada de acoplamiento y relaciones**, asegurando que la documentación reflejara fielmente la lógica de tu código en C#.
- **Enfoque analítico y metódico:** Al estructurar el manual técnico, mantuviste una postura crítica para exigir que el contenido se apegara estrictamente a los requerimientos de la consigna oficial, descartando información redundante o innecesaria.

2. Falencias observadas

- **Confusión operativa inicial con herramientas de archivos:** Presentaste dudas prácticas al sincronizar archivos locales con la estructura de directorios remota (como la ubicación física de la carpeta `docs` y su visibilidad en GitHub Desktop frente a la web).
- **Dependencia de asistencia estructurada:** Requeriste guía paso a paso para la organización y jerarquización de los apartados del manual técnico exigidos por la cátedra, lo que evidencia que la planificación documental global aún requiere mayor autonomía.

3. Diagnóstico del Hito 3

- **Programación Orientada a Objetos (POO):** *Comprensión demostrada.* Comprendiste la necesidad de aislar responsabilidades y modularizar clases en C#.

- **Clases y Objetos:** *Comprensión demostrada.* Identificaste correctamente la separación entre las plantillas de clases (ej. `Personaje`, `Objeto`) y las instancias creadas en el código.
- **Constructores:** *Evidencia insuficiente en la conversación directa.* No se registraron aportes o preguntas formuladas por ti sobre la sobrecarga o llamada explícita de constructores, por lo que se evalúa estrictamente en base a la consigna general integrada.
- **Encapsulamiento:** *Evidencia insuficiente en la conversación directa.* No desarrollaste explicaciones propias sobre modificadores de acceso durante el intercambio, aunque aceptaste su aplicación técnica en el manual.
- **Herencia:** *Comprensión demostrada.* Al analizar las relaciones de acoplamiento, reconociste la estructura jerárquica de clases base y derivadas (relación ES-UN).
- **Polimorfismo:** *Comprensión demostrada.* Comprendiste el propósito de invocar comportamientos dinámicos mediante referencias a clases base.
- **Interfaces:** *Comprensión parcial.* No implementaste activamente una interfaz en tus intervenciones directas; su inclusión provino de la estructuración general del manual para cumplir con el temario del Hito 3.
- **Listas genéricas (`List<T>`):** *Comprensión demostrada.* Reconociste su rol como contenedores dinámicos en componentes como el inventario.
- **LINQ:** *Comprensión demostrada.* Identificaste la utilidad de expresiones y consultas para filtrar y ordenar colecciones de datos.

4. Mi forma de aprender

- **Aprendizaje iterativo y de validación constante:** Tu metodología consistió en verificar paso a paso cada avance técnico (desde la creación de carpetas y repositorios hasta la revisión del manual en PDF), solicitando confirmación directa antes de avanzar al siguiente punto.
- **Orientación a entregizables académicos:** Tus decisiones estuvieron enfocadas en cumplir de manera estricta y limpia con los requerimientos formales de la rúbrica universitaria (Moodle y GitHub), priorizando la practicidad frente a herramientas teóricas accesorias.

5. Plan de estudio (Prioridades antes del examen)

1. **Implementación práctica de Interfaces:**
    - *Ejercicio propuesto:* Crear una interfaz `IUpgradeable` e implementarla de forma independiente en dos clases distintas de tu proyecto que no compartan la misma jerarquía de herencia.
2. **Uso avanzado de Consultas LINQ (`GroupBy` y `Aggregate`):**
    - *Ejercicio propuesto:* Escribir una consulta LINQ que agrupe los elementos de tu inventario por categoría y calcule el total acumulado de peso por cada grupo.
3. **Control de Excepciones en Propiedades Encapsuladas:**

    - *Ejercicio propuesto:* Modificar una propiedad con `set` privado para que lance una excepción explícita (`ArgumentException`) si se intenta asignar un valor inválido (por ejemplo, salud negativa).
4. **Sobrecarga de Constructores y uso de `base`:**
    - *Ejercicio propuesto:* Escribir dos constructores sobrecargados en una clase derivada que utilicen la palabra clave `base` para inicializar diferentes firmas de la clase padre.
5. **Polimorfismo con operadores de verificación de tipos (`is` / `as`):**
    - *Ejercicio propuesto:* Recorrer una lista polimórfica de tipo `Personaje` y utilizar el operador `is` para ejecutar un método exclusivo de la clase derivada `Robot` sin generar errores de tipo.

6. Conclusión

Has demostrado un dominio sólido en la gestión de repositorios en GitHub, la organización de proyectos en C# y la comprensión conceptual de los pilares de la POO (herencia, polimorfismo y colecciones). Los aspectos que requieren mayor refuerzo práctico de forma independiente son la implementación de contratos mediante interfaces y el manejo avanzado de consultas LINQ complejas. Con la consolidación de este manual técnico y la revisión de estos puntos, cuentas con la base necesaria para afrontar con éxito el examen del Hito 3.